

Project Title:

Development of a Centralized Email Sending Service (Mail API)

1. Objective

The goal of this project is to design and implement a centralized service that allows automated email sending from various external systems (e.g., PowerShell scripts, Excel files, web applications).

The service should provide a simple and standardized interface to submit email data and handle the delivery via an existing SMTP relay server.

2. Functional Requirements

The system must provide an HTTP-based API (REST API) with at least one endpoint:

Endpoint: Send Email

- Method: POST
- URL: /send-mail

Input Parameters (JSON):

- to (string or array) → recipient email address(es)
- subject (string) → email subject
- body (string) → email content (plain text and/or HTML)
- isHtml (boolean, optional) → indicates if body is HTML
- attachments (optional) → files such as PDF, images, etc.

Behavior:

- Validate incoming data
 - Generate an email
 - Send the email using the configured SMTP relay
 - Return a response indicating success or failure
-

3. Technical Requirements

- The service should be implemented using one of the following:
 - PHP (recommended: PHPMailer)
 - Python (recommended: FastAPI or Flask)
- Communication format: JSON
- Support for file uploads (preferably via multipart/form-data)
- Configurable SMTP settings (host, port, credentials if needed)

- Logging of all email requests (success and failure)
-

4. Non-Functional Requirements

- The system must be easy to use from:
 - PowerShell scripts
 - Excel (VBA or HTTP requests)
 - Clear and structured error handling
 - Maintainable and readable code
 - Basic security mechanism (e.g., API key)
-

5. Suggested Architecture

- REST API (receives requests)
 - Email service layer (handles email creation and sending)
 - SMTP relay (already provided)
 - Optional: configuration file (.env or similar)
-

6. Implementation Tips

Step 1: Basic API

- Create a simple API endpoint /send-mail
 - Accept JSON input
 - Return a basic response
-

Step 2: Email Sending

- Integrate a mail library:
 - PHP: PHPMailer
 - Python: smtplib or FastAPI-compatible libraries
- Test email sending locally

💡 Tip: Use a local test SMTP server like MailHog

to simulate email sending without actually delivering emails.

Step 3: Validation

- Check required fields (to, subject, body)
 - Validate email format
 - Handle missing or invalid data
-

Step 4: Attachments

- Implement file upload handling
 - Limit file size
 - Attach files to the email
-

Step 5: Configuration

- Externalize SMTP settings (e.g., .env file)
 - Allow switching between:
 - Local test SMTP (development)
 - Real SMTP relay (production)
-

Step 6: Logging

- Log:
 - Timestamp
 - Recipient
 - Status (success/failure)
 - Error messages
-

Step 7: Security (Basic)

- Add API key validation:
 - Request must include a header (e.g., X-API-KEY)
 - Reject unauthorized requests
-

7. Testing Requirements

The system must be tested with:

- PowerShell script sending a request
- Excel (VBA or HTTP request)
- Valid and invalid input data

- Emails with and without attachments
-

8. Deliverables

- Source code of the application
 - Configuration file example
 - API documentation (endpoint, parameters, examples)
 - Example requests (PowerShell / Excel)
 - Short technical documentation:
 - Setup instructions
 - How to run locally
 - How to switch to production SMTP
-

9. Optional Enhancements (Bonus)

- Queue system for delayed sending
 - Retry mechanism for failed emails
 - Web interface for testing
 - Rate limiting
 - Multiple recipients (CC, BCC)
-

10. Project Goal

At the end of the project, a working and reusable email service should be available that can be easily integrated into different systems to send automated emails in a standardized and reliable way.